City Mobility Platform — Teacher Access Guide

Student	Subir Saha
MID	946898
Course	Advanced Data Management and Decision Support Systems
University	University of Milano-Bicocca | A.Y. 2025/2026
Instructors	Andrea Campagner | Gloria Lopiano

---

1. How to Access This Submission

There are two ways to access the project. Both contain identical files.

Option 1 — GitHub Repository (Recommended)

`Link: https://github.com/subirsaha452-commits/city-mobility-platform_adm`

1.	Open the link in any browser.
2.	All source code is visible and readable directly in the browser — no download needed.
3.	Click any .py file to read the code with syntax highlighting.
4.	Click any folder name to browse its contents.

Option 2 — ZIP File

The ZIP file ADM_Subir_Saha_946898.zip is included alongside this guide.

1.	Right-click the ZIP and select Extract All (Windows) or double-click (Mac).
2.	Open the extracted folder ADM_New_Project/.
3.	Open TEACHER_GUIDE.pdf (this file) to navigate the project.

---

2. Project Folder Structure

```
ADM_New_Project/
  main.py                       Single entry point for all commands
  requirements.txt              Python dependencies
  TEACHER_GUIDE.pdf             This file
  README.md                     Technical overview

  PostgreSQL/                   Relational model
    setup_postgres.py           Creates 4 tables + indexes, loads data
```

```
  queries_postgres.py            Q1 Q2 Q3 Q4 in SQL

MongoDB/                         Document model
  setup_mongo.py                 Creates collections + indexes, loads data
  queries_mongo.py               Q1 Q2 Q3 Q4 in aggregation pipeline

Neo4j/                           Graph model
  setup_neo4j.py                 Creates User / Station / Trip nodes + edges
  queries_neo4j.py               Cypher: reachable stations + top-3 stations

Spark/                           Spark analytics
  spark_query2.py                Spark implementation of Query 2
  spark_graphframes.py           PageRank + Connected Components

Schema_Evolution/                Schema evolution
  schema_evolution.py            Adds battery_level to BATTERY events

Benchmark/                       Scalability benchmark
  benchmark.py                   Runs all queries across dataset sizes
  benchmark_results.csv          Actual measured timings

Reports/                         Deliverable documents
  ADM_Project_Report_Subir_Saha_946898.pdf   Main project report (24 pages)
  generate_report_v2.py          Report generator script

app/                             Database connection helpers
  postgres/db.py                 PostgreSQL connection (pg8000)
  mongo/db.py                    MongoDB connection (PyMongo)
  neo4j/db.py                    Neo4j connection (bolt://)
  utils/data_generator.py        Synthetic data generator
```

---

3. Assignment Requirements Coverage

Assignment Requirement	File(s)	Status
Relational schema (PostgreSQL)	PostgreSQL/setup_postgres.py	Complete
Document schema (MongoDB)	MongoDB/setup_mongo.py	Complete
Q1-Q4 in relational model	PostgreSQL/queries_postgres.py	Complete
Q1-Q4 in document model	MongoDB/queries_mongo.py	Complete
Graph model (Neo4j)	Neo4j/setup_neo4j.py	Complete
Graph queries (Cypher)	Neo4j/queries_neo4j.py	Complete
Schema evolution	Schema_Evolution/schema_evolution.py	Complete
Spark Query 2	Spark/spark_query2.py	Complete
GraphFrames PageRank	Spark/spark_graphframes.py	Complete
GraphFrames Connected Components	Spark/spark_graphframes.py	Complete
Scalability benchmark	Benchmark/benchmark.py	Complete
Performance discussion	Reports/ADM_Project_Report_Subir_Saha_946898.pdf	Complete

---

4. Reading the Report (No Installation Needed)

Open Reports/ADM_Project_Report_Subir_Saha_946898.pdf in any PDF viewer. The report is 24 pages and covers:

	Data model design and justification (relational vs. document vs. graph)
	All four queries with SQL and aggregation pipeline implementation and results
	Benchmark results with charts showing actual measured timings
	Schema evolution analysis for both PostgreSQL and MongoDB
	Spark and GraphFrames results (PageRank scores, Connected Components)
	Partitioning and replication discussion
	Conclusion and comparative analysis of all three database systems

---

5. Running the Project (Optional)

Requirements:

Requirement	Version	Notes
Python	3.11	
PostgreSQL	15+	Default port 5432
MongoDB	6+	Default port 27017
Neo4j Desktop	5+	Default bolt port 7687 password: 123456789
Java	11	Required for Spark and GraphFrames only

Commands:

Command	What it does
python main.py setup	Create all schemas and load default dataset
python main.py queries	Run all queries: PostgreSQL + MongoDB + Neo4j
python main.py spark	Run Spark Query 2 on MongoDB data
python main.py graphframes	Run GraphFrames PageRank and Connected Components
python main.py evolve	Run schema evolution (add battery_level)
python main.py benchmark --quick	Run quick 3-configuration scalability benchmark

---

6. Key Results at a Glance

Benchmark — Wall-clock seconds

Configuration	PG Q1	PG Q2	PG Q3	PG Q4	Mongo Q1	Mongo Q2	Mongo Q3	Mongo Q4
Small (1k users / 10k trips / 2 ev)	0.24	0.02	2.01	0.13	4.29	0.38	0.03	0.03
Medium (10k / 50k / 5 ev)	3.36	0.24	91.5	2.59	20.94	2.52	0.06	0.26
Large (50k / 100k / 10 ev)	6.87	1.19	373s	1.43	37.94	7.79	0.09	0.61

PostgreSQL wins Q1 and Q2 (JOIN-heavy). MongoDB wins Q3 and Q4 — up to 4,100x faster on Q3 at large scale.

GraphFrames PageRank — Top 3 Stations

Rank	Station	City	PageRank Score
1	Padua Porto	Padua	1.149316
2	Milan Stadio	Milan	1.122747
3	Verona Sud	Verona	1.089738

Connected Components

All 50 stations form 1 connected component — every station is reachable from every other station.

Schema Evolution

Database	Operation	Result
PostgreSQL	ALTER TABLE events ADD COLUMN battery_level INT	250,010 rows back-filled
MongoDB	update_many with arrayFilters	94,432 documents updated